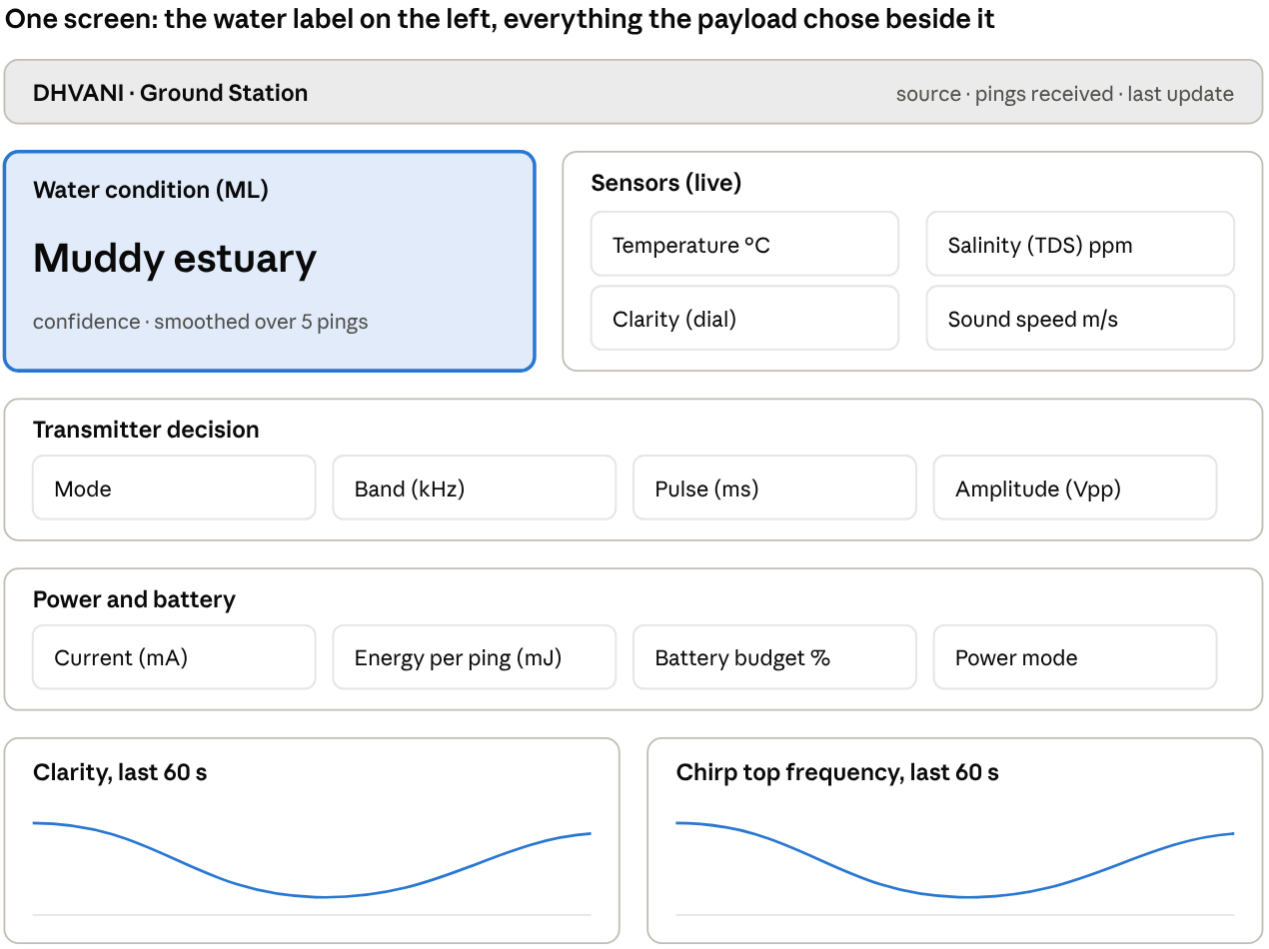
# DHVANI Ground-Station Dashboard – How to Build It

Oct 3, 2026 · @harshit

## What the dashboard shows

One screen on the laptop, refreshed twice a second, built in Python with Streamlit. It only reads the board's serial output — it never sends anything back.




dashboard layout · one screen, five areas

- **Water condition (big card):** the ML model's label, coloured by class, with its confidence.
- **Sensors:** temperature, salinity, the clarity dial and the speed of sound the board worked out.
- **Transmitter decision:** what the payload chose for this ping — mode, band, pulse length, amplitude.

- **Power and battery:** real current, energy per ping, battery budget and NORMAL / ECONOMY / SURVIVE.
- **Two live graphs:** clarity and the chirp's top frequency over the last minute — turning the dial moves both lines together, which is the adaptation in one picture.

## How it works inside

A background thread reads the serial port into a rolling buffer; the screen redraws from that buffer every half second, so a slow redraw never drops a ping.

| Part | Job |
|---|---|
| Reader thread | Opens the COM port at 115200, reads one line per ping, parses it with a regex, appends a dictionary to the buffer. Retries every 2 s if the port is busy or unplugged. |
| Buffer | A `deque` holding the last 600 pings (about 2 minutes). Old pings fall off the end. |
| Model | `dhvani_model.joblib` from the ML guide, loaded once. Predicts on the last 5 pings and shows the majority label, so the card doesn't flicker. |
| Screen loop | Every 0.5 s: redraws the card, 12 numbers and 2 graphs from the newest pings. |
| Replay mode | Reads a saved `log.txt` instead of the port, so the dashboard can be built and filmed without the board. |

The serial line it parses (exactly as the firmware prints it):

```
AUD-LFM  T=24.4C c=1495 clarity=4092 (clear=3900) tds=803ppm f=2000-17000Hz
dur=4ms amp=1669 | I=11.70mA E=6.4mJ batt=86% NORMAL
```

## Setup

One folder, three files, one install command.

```
dhvani-dashboard/
├── app.py                  ← the dashboard (next section)
├── capture.py              ← saves raw serial lines to log.txt (for replay)
├── dhvani_model.joblib     ← from the ML guide (optional: the dashboard runs
without it)
└── requirements.txt
```

`requirements.txt`:

```
streamlit
pyserial
pandas
scikit-learn
joblib
```

Install: `pip install -r requirements.txt`

`capture.py` — record 5 minutes of real pings once, while turning the dial and adding salt, so you can rebuild and film the dashboard any time without the board:

```
import sys, time, serial

port = sys.argv[1]                                  # e.g. COM9
seconds = int(sys.argv[2]) if len(sys.argv) > 2 else 300
with serial.Serial(port, 115200, timeout=2) as ser, open("log.txt", "w") as
f:
    end = time.time() + seconds
    while time.time() < end:
        line = ser.readline().decode(errors="ignore").strip()
        if line:
            f.write(line + "\n")
```

Run: `python capture.py COM9 300`

## The full app.py

Copy this as-is. It works with the four water classes from the ML guide, and falls back to the raw label (or a warning) if the model uses different names or is missing.

```
import re, sys, time, threading, collections
from collections import Counter
import joblib, pandas as pd, serial, streamlit as st

SOURCE = sys.argv[1] if len(sys.argv) > 1 else "COM9"     # or replay:log.txt
FEATURES = ["temp_c", "tds_ppm", "clarity"]
```

```
NAMES = {"clear_fresh": "Clear freshwater", "clear_reef": "Clear shallow
reef",
         "muddy_river": "Muddy river", "muddy_estuary": "Muddy estuary"}
COLOURS = {"clear_fresh": "#2563EB", "clear_reef": "#0D9488",
           "muddy_river": "#B45309", "muddy_estuary": "#7C2D12"}
LINE = re.compile(                                      # matches AUD-LFM and
"US -BPSK"
    r"(\w+ ?-\w+)\s+T=(-?[\d.]+)C c=(\d+) clarity=(\d+) \(clear=\d+\) tds=
(\d+)ppm "
    r"f=(\d+)-(\d+)Hz dur=(\d+)ms amp=(\d+) \| I=([\d.]+)mA E=([\d.]+)mJ "
    r"batt=(\d+)% (\w+)")

def parse(m):
    g = m.groups()
    return {"t": time.time(), "mode": g[0].replace(" ", ""), "temp_c":
float(g[1]),
            "c": int(g[2]), "clarity": int(g[3]), "tds_ppm": int(g[4]),
            "f_lo": int(g[5]), "f_hi": int(g[6]), "dur": int(g[7]), "amp":
int(g[8]),
            "i_ma": float(g[9]), "e_mj": float(g[10]), "batt": int(g[11]),
            "power": g[12]}

@st.cache_resource
def start_reader(source):
    buf = collections.deque(maxlen=600)                 # about 2 minutes of
pings
    def lines():
        if source.startswith("replay:"):
            while True:                                 # loop a saved log
forever
                for line in open(source[7:], encoding="utf-8",
errors="ignore"):
                    yield line
                    time.sleep(0.25)
        else:
            while True:
                try:
                    with serial.Serial(source, 115200, timeout=2) as ser:
                        while True:
                            yield ser.readline().decode(errors="ignore")
                except serial.SerialException:
                    time.sleep(2)                       # port busy or
unplugged: retry
    def run():
        for line in lines():
            m = LINE.search(line)
```

```
            if m:
                buf.append(parse(m))
    threading.Thread(target=run, daemon=True).start()
    return buf

@st.cache_resource
def load_model():
    try:
        return joblib.load("dhvani_model.joblib")
    except FileNotFoundError:
        return None

st.set_page_config(page_title="DHVANI Ground Station", page_icon="🔊",
layout="wide")
buf, model = start_reader(SOURCE), load_model()
st.markdown("## DHVANI · Ground Station")
view = st.empty()

while True:
    with view.container():
        if not buf:
            st.info(f"Waiting for data from {SOURCE}…")
        else:
            rows = list(buf)
            r, df = rows[-1], pd.DataFrame(rows)
            left, right = st.columns([1, 2])

            with left:                                    # water condition
card
                if model is None:
                    st.warning("dhvani_model.joblib not found — showing data
only")
                else:
                    recent = df.tail(5)[FEATURES]
                    label = Counter(model.predict(recent)).most_common(1)[0]
[0]
                    conf = model.predict_proba(recent.tail(1)).max()
                    st.markdown(
                        f"<div style='background:{COLOURS.get(label,
'#334155')};color:white;"
                        f"padding:24px;border-radius:12px'>"
                        f"<div style='font-size:14px;opacity:.85'>Water
condition (ML)</div>"
                        f"<div style='font-size:34px;font-weight:700'>
{NAMES.get(label, label)}</div>"
                        f"<div style='font-size:13px;opacity:.85'>{conf:.0%}
```

```
sure · "
                         f"smoothed over last 5 pings</div></div>",
unsafe_allow_html=True)

            with right:                                    # 12 live numbers
                a, b, c, d = st.columns(4)
                a.metric("Temperature", f"{r['temp_c']:.1f} °C")
                b.metric("Salinity (TDS)", f"{r['tds_ppm']} ppm")
                c.metric("Clarity (dial)", r["clarity"])
                d.metric("Sound speed", f"{r['c']} m/s")
                a, b, c, d = st.columns(4)
                a.metric("Mode", r["mode"])
                b.metric("Band", f"{r['f_lo']/1000:.0f}–{r['f_hi']/1000:.1f}
kHz")
                c.metric("Pulse", f"{r['dur']} ms")
                d.metric("Amplitude", f"{2 * r['amp'] * 3.3 / 4095:.2f} Vpp")
                a, b, c, d = st.columns(4)
                a.metric("Current", f"{r['i_ma']:.2f} mA")
                b.metric("Energy / ping", f"{r['e_mj']:.1f} mJ")
                c.metric("Battery budget", f"{r['batt']} %")
                d.metric("Power mode", r["power"])

            now = time.time()                              # two live graphs
            last = df[df.t > now - 60].copy()
            last["seconds"] = (last.t - now).round(1)
            g1, g2 = st.columns(2)
            g1.caption("Clarity, last 60 s")
            g1.line_chart(last, x="seconds", y="clarity", height=220)
            g2.caption("Chirp top frequency (Hz), last 60 s")
            g2.line_chart(last, x="seconds", y="f_hi", height=220)

            st.caption(f"Source {SOURCE} · {len(rows)} pings in buffer · last
update "
                       f"{now - r['t']:.1f} s ago · clarity is simulated by a
dial")
    time.sleep(0.5)
```

## Running it

Two ways to run: live from the board, or replaying a saved log when the board isn't with you.

**Live from the board:**

1. Plug in the board, press RESET, check pings are running (LD1 flipping).

2. Close the CubeIDE serial console and PuTTY — only one program can hold the COM port.
3. Find the port in Device Manager → Ports (e.g. COM9).
4. Run: `streamlit run app.py -- COM9`
5. The browser opens at `localhost:8501`. Numbers appear within a second.

**Replay mode (no board needed):**

1. Once, with the board: `python capture.py COM9 300` while you turn the dial end to end and stir in salt.
2. Afterwards, anywhere: `streamlit run app.py -- replay:log.txt`
3. The dashboard plays the saved pings at 4 per second, looping forever — good for building the UI and for filming.

The `--` before the port matters: everything after it is passed to `app.py` rather than to Streamlit.

## Making it look good and recording it

A dark theme, a full-screen browser and a slow dial turn make a clean 25-second clip for the 2:15–2:40 video slot.

**Dark theme** — create `.streamlit/config.toml` next to `app.py`:

```
[theme]
base = "dark"
primaryColor = "#14B8A6"
font = "sans serif"
```

**Recording the clip:**

1. Browser full screen (F11), zoom 110–125% so the numbers read on a phone.
2. Start a screen recording: Windows **Win + Alt + R**, or OBS.
3. Turn the dial slowly from clear to muddy (5–6 s), pause, then stir salt into the glass.
4. On screen: the card changes colour and label, Band and Pulse change, both graphs move together, Amplitude rises with the salt.
5. Best shot: film the laptop and the board side by side with a phone, so judges see the dial turn and the dashboard react in the same frame.

**Voiceover for this clip** (already in the v5 script): *"On the surface, a laptop dashboard shows everything the payload is doing, live. A machine-learning model reads the sensor data and tells the operator what kind of water the robot is in."*

# Checklist and common problems

Tick these off, and use the table when something on screen looks wrong.

- [ ] `pip install -r requirements.txt` done
- [ ] Live mode shows numbers within a second of starting
- [ ] `log.txt` captured (5 min, dial end to end + salt) and replay mode works
- [ ] `dhvani_model.joblib` copied in; the card shows a label and confidence
- [ ] Turning the dial moves both graphs and changes Band and Pulse
- [ ] Salt raises TDS and Amplitude
- [ ] Dark theme on, 25-second clip recorded

| Problem | Cause | Fix |
|---|---|---|
| "Waiting for data…" forever | Port held by another program, or wrong COM number | Close CubeIDE console / PuTTY; check Device Manager; unplug and replug |
| Numbers frozen, "last update" growing | Board stopped or USB unplugged | RESET the board; the reader reconnects by itself within 2 s |
| Card says model not found | `dhvani_model.joblib` not in the folder | Copy it next to `app.py` and restart Streamlit |
| Card shows a strange label name | Model trained with different class names | Add those names to `NAMES` and `COLOURS` |
| Card flickers between two labels | Dial sitting on a class boundary | Move the dial firmly into a zone; or smooth over 10 pings instead of 5 |
| Power mode stuck on SURVIVE | Board's demo battery budget used up | Press RESET on the board |
| Graphs empty | No pings in the last 60 s | Check the board is pinging |